Spaceship Titanic

Midterm Report

EARIN - Introduction to Artificial Intelligence

Tibor Sterle, Francesco Gaibazzi

May 2026

1. Problem Description

The Spaceship Titanic project is a binary classification task based on the Kaggle competition of the same name. In the year 2912, the interstellar passenger ship Spaceship Titanic collided with a spacetime anomaly during its maiden voyage, causing roughly half of its 13,000 passengers to be transported to an alternate dimension. The goal is to predict, from passenger records recovered from the ship's computer system, whether a given passenger was transported (True) or not (False).

This is a supervised learning problem with a binary target variable. The two classes are approximately balanced, with about 50% of passengers transported and 50% not, which makes evaluation straightforward and avoids the need for class-imbalance corrections. The main evaluation metric used by the Kaggle competition is prediction accuracy.

2. Dataset

The dataset comes from the Kaggle Spaceship Titanic competition and contains two CSV files:

- train.csv: 8,693 labelled passenger records used for training and validation.
- test.csv: 4,277 passenger records without labels, used for the final Kaggle submission.

The main features are:

- HomePlanet, Destination: the planet the passenger departed from and is travelling to (categorical).
- CryoSleep, VIP: boolean flags indicating whether the passenger was in suspended animation or had VIP status.
- Cabin: a combined field in the format deck/number/side, which is split during preprocessing into three separate features: Deck, CabinNum, and Side.
- Age: the passenger's age (numerical).
- RoomService, FoodCourt, ShoppingMall, Spa, VRDeck: amounts spent at each onboard amenity (numerical). These five columns are also combined into a single engineered feature called TotalSpending.

Several columns contain missing values, which are filled during preprocessing using the median for numerical features and the mode for categorical ones. All imputation statistics are computed only on the training set to avoid data leakage.

3. Exploratory Data Analysis

Before building the models, an exploratory data analysis was carried out to understand the data and identify which features are most useful for predicting the target.

3.1 Numerical Features

The histograms below show the distribution of numerical features separated by the Transported label. The most notable pattern is that passengers who spent nothing on amenities are much more likely to have been transported. This is strongly linked to the CryoSleep feature: passengers in cryo-sleep cannot use the ship's services and are almost always transported. As a result, TotalSpending turns out to be one of the most useful features in the dataset.

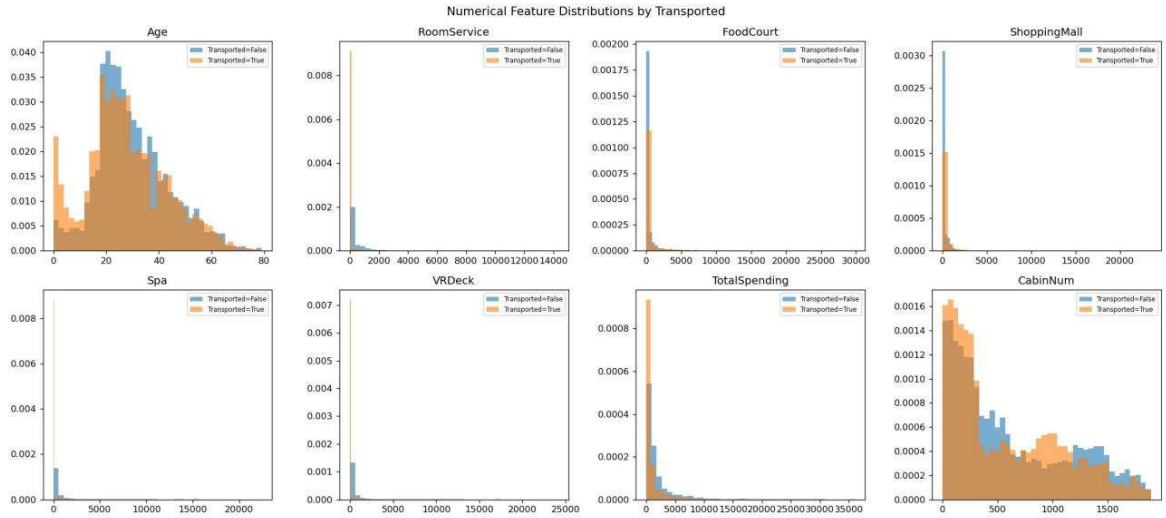


Figure 1 - Numerical feature distributions by Transported (Random Forest EDA)

3.2 Categorical Features

The count plots below show how categorical features relate to the target. CryoSleep is the most predictive single feature: almost all passengers in cryo-sleep were transported. HomePlanet also shows a clear pattern, with Europa passengers being transported at a higher rate than those from Earth. VIP status and Destination show weaker associations with the target.

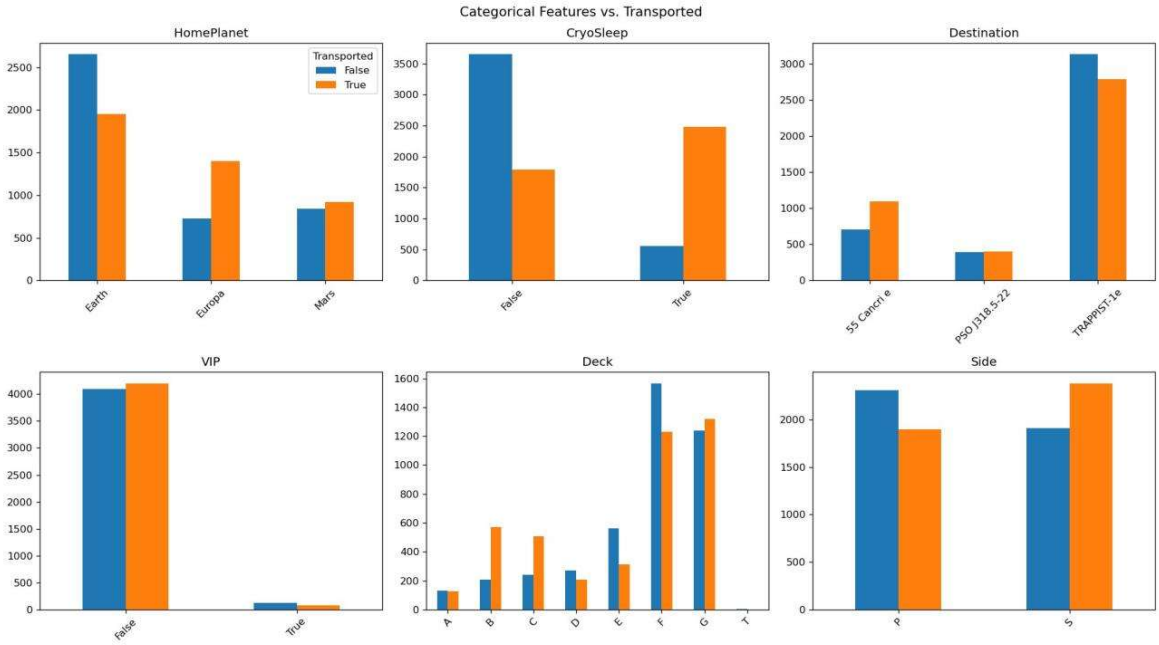


Figure 2 - Categorical features vs. Transported (Random Forest EDA)

4. Proposed Methods

Three machine learning models were implemented and compared: Random Forest, XGBoost, and a Multilayer Perceptron (MLP) neural network. All three share the same preprocessing pipeline.

4.1 Project Evolution

The initial project plan included a linear regression model as one of the three algorithms. During implementation, we replaced it with XGBoost, a gradient boosted tree model, because linear regression is not well suited for binary classification tasks and would require additional workarounds such as thresholding and probability calibration. XGBoost is a more appropriate and competitive baseline for this type of problem.

In addition, 5-fold cross-validation was introduced for all three models. The initial plan mentioned cross-validation but it was not consistently applied in the first version of the code. It has now been added to all models to provide a more reliable estimate of generalisation performance and to make the comparison between models more robust.

4.2 Preprocessing Pipeline

The preprocessing steps applied to all models are the same:

- Missing values are filled using the median for numerical columns and the mode for categorical columns, computed on the training set only.
- Cabin is split into three sub-features: Deck, CabinNum, and Side. The five amenity columns are summed into TotalSpending.
- All categorical and boolean columns are label-encoded as integers using scikit-learn's LabelEncoder. Each unique category is assigned a fixed integer in alphabetical order (for example, Earth=0, Europa=1, Mars=2 for HomePlanet).

- For the MLP only, all features are standardised using StandardScaler fitted on the training set.
- An 80/20 stratified train/validation split is used for all models. 5-fold stratified cross-validation is used to estimate generalisation accuracy.

4.3 Random Forest

An ensemble of decision trees from scikit-learn, where each tree is trained on a random sample of the data and a random subset of features. The final prediction is made by majority vote. A grid search was run over four combinations of n_estimators (200, 300, 500) and max_depth (15, 20, None) to find the best configuration. Limiting max_depth helps reduce overfitting. The best model is then evaluated with 5-fold cross-validation.

4.4 XGBoost

A gradient boosted tree model (XGBClassifier) that builds trees one at a time, where each new tree corrects the errors of the previous ones using gradient descent on the log-loss objective. The main settings are: n_estimators=500, max_depth=6, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8. Early stopping (early_stopping_rounds=30) is used during training to stop when validation loss stops improving, which speeds up training without hurting accuracy. A separate model instance without early stopping is used for cross-validation, because scikit-learn's cross_val_score does not support passing an eval_set.

4.5 MLP Neural Network

A fully-connected neural network built in PyTorch. The architecture has a BatchNorm1d input layer, three hidden layers with 256, 128, and 64 units using ReLU activations and Dropout (p=0.3) for regularisation, and a final sigmoid output. Training uses the Adam optimiser with binary cross-entropy loss and a ReduceLROnPlateau scheduler that lowers the learning rate when progress stalls. Early stopping with patience=15 monitors the validation loss and saves the best weights, stopping training around epoch 85 instead of running the full 100 epochs. A small grid search is performed over three configurations of hidden sizes, learning rate, and dropout.

5. Intermediate Results

All three models were trained and evaluated on the same 80/20 stratified split. The table below summarises the key metrics on the validation set.

Model	Accuracy	F1-Score	ROC-AUC	CV Accuracy
Random Forest	0.807	0.808	0.900	~0.806
MLP (Neural Network)	0.809	0.816	0.906	~0.803
XGBoost	0.817	0.820	0.914	~0.815

Table 1 - Validation set performance for all three models

XGBoost achieves the best results across all metrics, with a validation accuracy of 0.817 and ROC-AUC of 0.914. The MLP follows closely with 0.809 accuracy and 0.906 AUC, showing a clear improvement compared to the initial version thanks to early stopping restoring the best weights during training. Random Forest is slightly behind with 0.807 accuracy and 0.900 AUC. The cross-validation scores are consistent with the validation set results, confirming that the models are stable and not overfitting.

5.1 Confusion Matrices

The confusion matrices below show the number of correct and incorrect predictions for each model on the validation set, which contains approximately 1,739 samples.

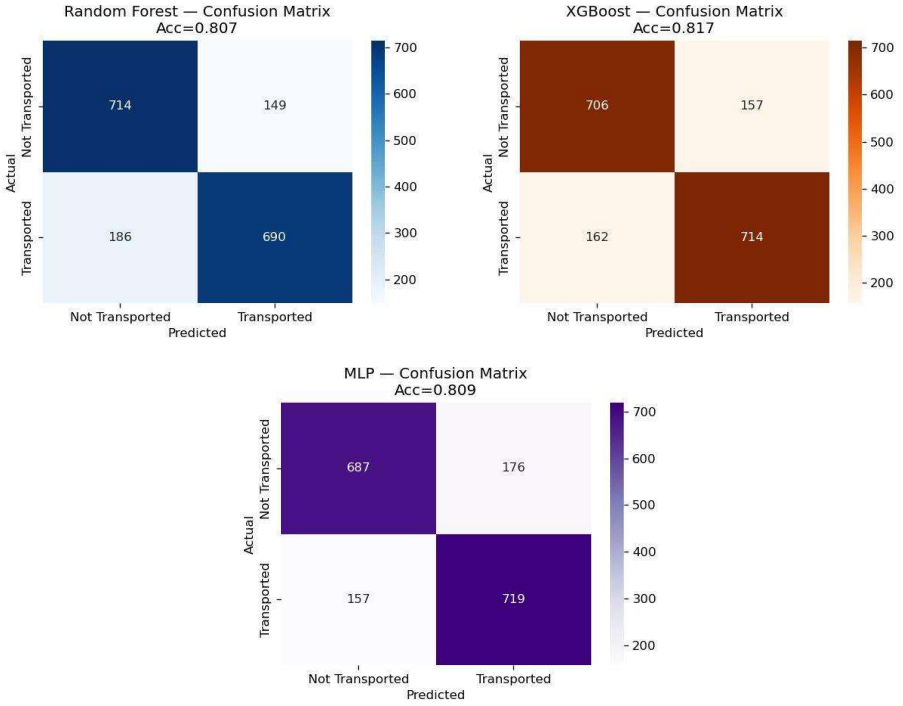


Figure 3 - Confusion matrices: Random Forest (left), XGBoost (centre), MLP (right)

All three models show a balanced error distribution between false positives and false negatives. XGBoost makes the fewest total errors (319 out of 1,739), followed by MLP (333) and Random Forest (335).

5.2 ROC Curves

The ROC curves confirm the ranking seen in the accuracy results. XGBoost reaches the highest AUC (0.914), followed by MLP (0.906) and Random Forest (0.900). All three curves are well above the random baseline.

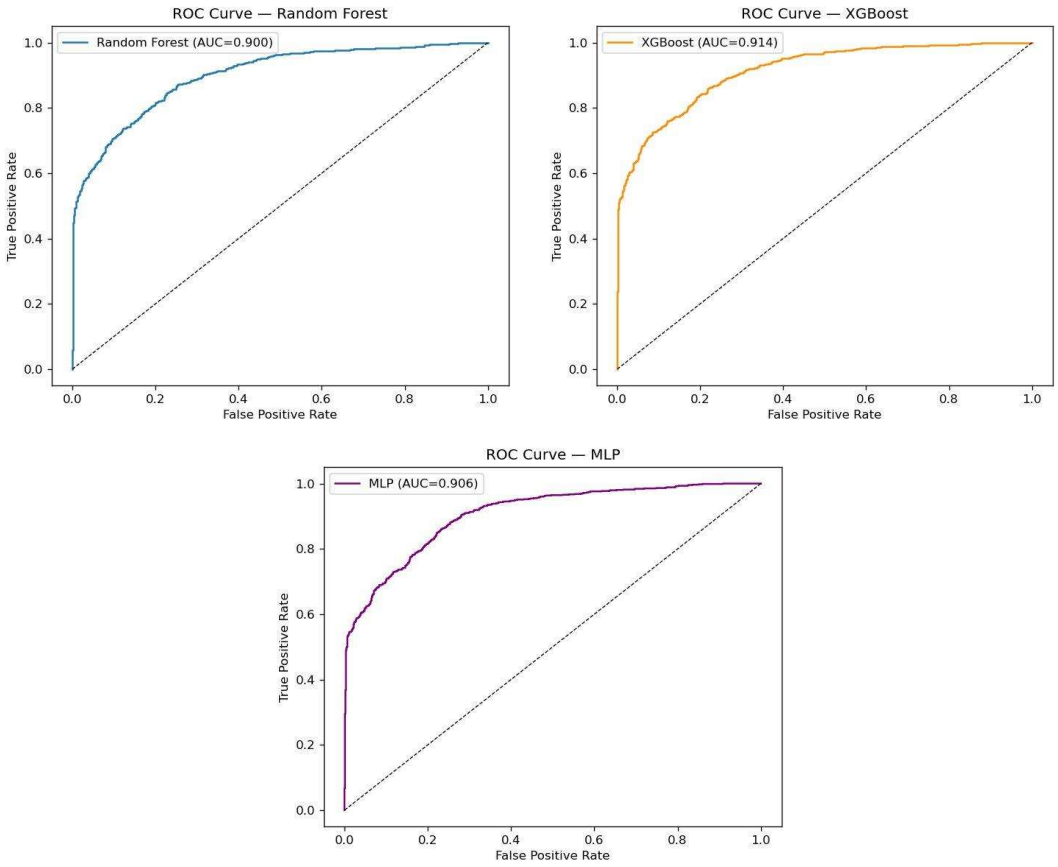


Figure 4 - ROC curves: Random Forest (top-left), XGBoost (top-right), MLP (bottom)

5.3 Feature Importances

The feature importance charts show which features each tree-based model relies on most. The two models give quite different rankings, which reflects how each one works internally.

Random Forest spreads importance more evenly across features. TotalSpending and CabinNum rank highest, followed by Age, Spa, RoomService, CryoSleep, FoodCourt, and VRDeck at similar levels. This suggests the model combines many different signals to make predictions.

XGBoost gives a much larger weight to CryoSleep (about 0.32), with HomePlanet and TotalSpending as the next most important features (around 0.12 to 0.13 each). This concentrated profile reflects the strong non-linear relationship between CryoSleep and spending: passengers in cryo-sleep spend nothing and are almost always transported, which XGBoost captures very effectively.

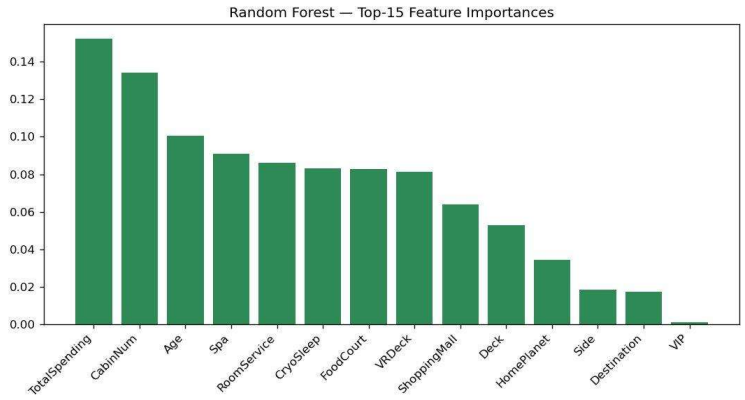


Figure 5 - Random Forest top-15 feature importances

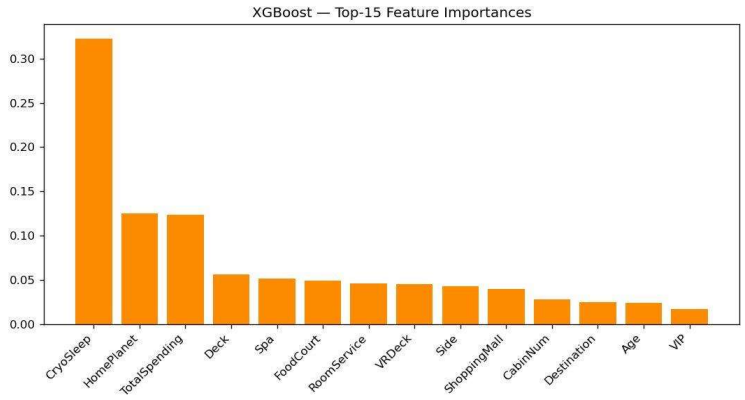


Figure 6 - XGBoost top-15 feature importances

5.4 MLP Training Curve

The loss curve below shows training and validation BCE loss across epochs. Both curves decrease steadily and stay close together throughout training, with no sign of overfitting. Early stopping triggered around epoch 85. The final validation loss is approximately 0.39.

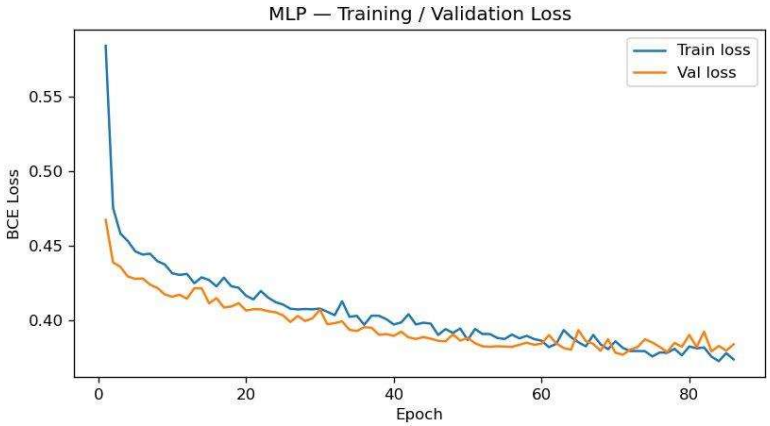


Figure 7 - MLP training and validation loss curve

6. Challenges and Solutions

XGBoost early stopping incompatibility with cross_val_score.

The early_stopping_rounds parameter requires an eval_set to be passed during training. Since scikit-learn's cross_val_score manages the training call internally and does not support eval_set, using a model with early stopping in the cross-validation loop caused a ValueError. The solution was to use two separate model instances: one with early stopping for the main training run, and one without for the cross-validation loop.

XGBoost API version incompatibility.

The early_stopping_rounds argument was initially passed to .fit() as expected in newer XGBoost versions, but the installed version required it to be set in the constructor instead. The fix was to move the parameter from the .fit() call to the XGBClassifier constructor, which works across versions.

MLP training time.

The original MLP ran all epochs unconditionally, even after the model had already converged. With a grid search over three configurations plus 5-fold cross-validation, this meant many unnecessary training epochs. Early stopping with patience=15 and best-weight checkpointing was added, reducing the effective number of epochs per run to around 85 and cutting total execution time noticeably.

Dataset path configuration.

A FileNotFoundError was raised because the CSV files were placed in the project root instead of the expected data/ subdirectory. The issue was fixed by creating a data/ folder inside the codes/ directory and placing train.csv and test.csv there, matching the DATA_DIR path defined in all three scripts.

7. Potential Risks and Obstacles

- Performance ceiling: all three models are currently in the 0.807 to 0.817 accuracy range. The top scores on the Kaggle leaderboard are around 0.82 to 0.83, which means there is limited room for improvement without more advanced feature engineering or model architectures.
- Overfitting risk in Random Forest: the grid search includes max_depth=None (no depth limit), which can produce very deep trees that memorise the training data. While validation accuracy looks stable for now, this configuration may not generalise as well on the test set.
- MLP sensitivity to hyperparameters: the neural network is more sensitive to choices like learning rate and layer sizes than the tree-based models. The current small grid search may not find the best configuration.
- Reproducibility: PyTorch training is not fully deterministic, especially on GPU, due to non-deterministic operations in CUDA. Results may vary slightly between runs.
- Missing visualisations: the preliminary documentation calls for a combined ROC curve showing all three models on a single chart and an accuracy comparison bar chart. These have not been produced yet and will be included in the final report.